

MTasking

Manual Técnico

Materia: Construcción de Software

Docente: PhD. Franklin Parrales

Responsable: Daniel Mora Cantillo

Julio 2026

1. Descripción del sistema

MTasking es una aplicación web de gestión de tareas tipo Kanban desarrollada como proyecto académico. Permite crear proyectos, agregar tareas con prioridad y fecha límite, asignar responsables y hacer seguimiento del estado de avance.

Capa	Tecnología
Backend	PHP 8 sin frameworks (PDO, sesiones nativas)
Frontend	HTML + CSS + JavaScript Vanilla
Base de datos	MySQL (utf8mb4)
Servidor local	XAMPP — Apache en puerto 8083
Pruebas	PHPUnit via phpunit.phar — SQLite en memoria

2. Arquitectura

El sistema sigue el patrón MVC implementado de forma manual (sin framework). El flujo de una petición es el siguiente:

```
Browser → Apache (XAMPP :8083)
        → backend/index.php (CORS + JSON headers)
        → backend/routes/api.php (router con preg_match)
        → Controllers (AuthController / ProjectController / TaskController)
        → Models (User / Project / Task)
        → MySQL / SQLite (tests)
```

Estructura de carpetas

```
backend/
  config/database.php      ← Singleton PDO (MySQL prod / SQLite tests)
  controllers/             ← AuthController, ProjectController, TaskController
  exceptions/AppException.php ← Jerarquía de excepciones personalizadas
  models/                 ← User, Project, Task
  routes/api.php          ← Router manual
  index.php / .htaccess
frontend/
  html/ index.html dashboard.html project.html
  js/   auth.js projects.js tasks.js
  css/  style.css
database/schema.sql
tests/
  bootstrap.php DatabaseTestCase.php
  Unit/ UserModelTest ProjectModelTest TaskModelTest
```

3. Base de datos

Base de datos: mtasking. Archivo: database/schema.sql.

Tabla	Columnas principales
users	id, nombre, email (UNIQUE), password (bcrypt), created_at
projects	id, nombre, descripcion, user_id (FK→users), created_at
tasks	id, titulo, descripcion, estado ENUM(Pendiente/En progreso/Terminado), prioridad ENUM(Alta/Media/Baja), fecha_limite DATE, proyecto_id (FK), responsable_id (FK→users)
task_comments	id, task_id, user_id, contenido, created_at [pendiente]
task_history	id, task_id, user_id, estado_anterior, estado_nuevo, created_at [pendiente]
password_resets	id, email, token (UNIQUE), expires_at [pendiente]

4. API REST

Base URL: <http://localhost:8083/mtasking/backend/index.php>

Autenticación: cookie de sesión PHP (PHPSESSID). Todas las respuestas son JSON.

Método	Ruta	Descripción
POST	/auth/register	Registrar usuario
POST	/auth/login	Iniciar sesión
POST	/auth/logout	Cerrar sesión
GET	/auth/me	Usuario de la sesión activa
POST	/projects	Crear proyecto
GET	/projects	Listar proyectos del usuario
GET	/projects/{id}	Detalle de un proyecto
PUT	/projects/{id}	Editar proyecto
POST	/tasks	Crear tarea
GET	/projects/{id}/tasks	Listar tareas (filtros: ?estado=&prioridad=)
GET	/tasks/{id}	Detalle de una tarea
PUT	/tasks/{id}/status	Cambiar estado de tarea
DELETE	/tasks/{id}	Eliminar tarea
GET	/users	Listar usuarios (para selector de responsable)

5. Pruebas

14 tests unitarios sobre los modelos User, Project y Task. Usan SQLite en memoria para no depender de MySQL.

La constante TESTING = true (definida en tests/bootstrap.php) hace que getDB() retorne una instancia SQLite inyectada por DatabaseTestCase en lugar de conectar a MySQL. Cada test parte de una base de datos limpia.

```
# Ejecutar desde la raíz del proyecto  
php phpunit.phar
```

```
# Resultado esperado  
OK (14 tests, 28 assertions)
```

6. Instalación

1. Iniciar XAMPP (Apache puerto 8083 + MySQL)
2. Copiar backend/, frontend/, database/ a:
C:/xampp/htdocs/mtasking/
3. Importar database/schema.sql en phpMyAdmin
4. Abrir en el navegador:
<http://localhost:8083/mtasking/frontend/html/index.html>

Credenciales de BD en backend/config/database.php (host: localhost, db: mtasking, user: root, pass: vacía).